SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I _Lavanya.M (192321051)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

Answer:

Behaviour: Greedy Best-First Search (GBFS) ranks every frontier node only by the heuristic, f(n) = h(n), the straight-line distance to the destination. The drone therefore always flies toward whichever waypoint looks closest to the target, ignoring the cost already spent (wind, rain, flooded terrain).

Strengths:

- Very fast: it expands few nodes and needs little memory, which suits an urgent medicine delivery.
- Simple to re-run each time new information arrives.

Weaknesses under uncertainty:

- It is not optimal: a path that looks short may cross high-cost storm zones or flooded terrain.
- It is not complete in graphs with loops unless visited nodes are tracked, and it can be trapped in dead ends, e.g. a straight-line approach into a river or blocked valley.
- It ignores variable terrain and weather cost, so it can choose risky routes and lose the drone or cargo.

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

Answer:

A* ranks nodes by **f(n) = g(n) + h(n)**, where g(n) is the actual accumulated cost from the base (distance adjusted for terrain and weather) and h(n) is the straight-line estimate to the target.

- g(n) keeps the search honest: a route that is already expensive (strong headwind, flooded area) is penalised.
- h(n) pulls the search toward the goal so fewer nodes are expanded than in uniform-cost search.
- Because the straight-line distance never overestimates the real distance, it is **admissible**, so A* is guaranteed to return the lowest-cost path (and is complete).

- When a path is reported blocked, the node is removed or its cost set to infinity and A* re-plans from the drone's current position, automatically choosing the next cheapest route.

So A* balances "how much have I already paid" (g) against "how far do I still seem to have to go" (h), which gives routes that are both short and safe.

iii. Analyze the impact of heuristic accuracy on both algorithms

Answer:

- **Greedy Best-First Search:** its entire decision depends on h(n). A poor heuristic (e.g. straight-line distance across a flooded river) sends the drone confidently into dead ends. A very accurate heuristic makes it fast and nearly direct, but even then optimality is not guaranteed because g(n) is ignored.
- **A*:** if h(n) is admissible and consistent (never overestimates), the result is optimal. The closer h(n) is to the true cost, the fewer nodes are expanded. If h(n) = 0, A* becomes uniform-cost search (optimal but slow). If h(n) overestimates, A* becomes faster but may miss the best route.
- In this scenario straight-line distance ignores terrain and weather, so it underestimates the true cost. That is safe for A* (still optimal, just a bit slower) but harmful for greedy search. Improving h(n) with weather and terrain factors (while keeping it admissible) helps both.

iv. Discuss how dynamic changes (blocked paths) affect search performance

Answer:

- When a path is suddenly blocked, a previously computed plan becomes invalid and the drone must re-plan from its current location.
- **Greedy** re-plans very quickly, but it may keep choosing routes toward the same blocked region and can oscillate between dead ends.
- **A*** needs more computation per re-plan, but it uses real accumulated costs and so finds the next best valid route. Variants such as **D* Lite / Lifelong Planning A*** reuse earlier search results and update only the affected nodes, which greatly reduces re-planning time.
- Frequent changes increase the time and energy spent on re-planning, so the system should refresh only when the update affects the current route, and keep a safety margin (battery reserve) for detours.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

Answer:

Recommendation: A* (with an admissible weather-aware heuristic, using a dynamic variant such as D* Lite for re-planning).

- **Optimality and safety:** A* minimises the true cost (terrain, weather, risk), avoiding dangerous areas that greedy search would walk into.

- **Real-time constraints:** the heuristic keeps A* far faster than uninformed search, and incremental re-planning keeps delays small. If a decision is needed in a few milliseconds, weighted A* ($f = g + w \cdot h$, w slightly above 1) gives a near-optimal route much faster.
- **Robustness:** completeness guarantees that a route is found whenever one exists, which matters for emergency medicine delivery.
- Greedy search is only acceptable as a quick fallback when time is extremely limited, since its savings come at the cost of safety.

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

Answer:

Formulation:

- **State:** the complete set of signal timings (cycle length, green-phase split, offset) for all intersections.
- **Actions / neighbours:** change one or a few timing parameters, e.g. lengthen a green phase by 5 s or shift an offset.
- **Objective function:** minimise Cost = $w_1 \cdot$(total waiting time) + $w_2 \cdot$(fuel consumption) + $w_3 \cdot$(congestion/queue length), where the weights w_1, w_2, w_3 reflect city priorities.
- **Constraints:** minimum green time, pedestrian crossing time, safe cycle length, emergency-vehicle priority.

Why traditional search is inefficient:

- With hundreds of intersections and many timing choices each, the search space grows exponentially (about k^n states for n intersections with k options), so BFS, DFS or A* cannot enumerate it.
- There is no single goal state; we need the best configuration, and the path to it does not matter.
- Traffic changes continuously, so any exhaustive solution is outdated before it finishes.
- Intersections affect each other (a green wave on one road changes queues on the next), so the problem cannot be split into independent parts.

i. Explain how Hill Climbing would work in this scenario and identify its limitations

Answer:

- Start with the current (or a random) timing plan and compute its cost.

- Generate neighbouring plans by making small changes to the timings.
- Move to the neighbour with the best (lowest) cost and repeat; stop when no neighbour is better.

Advantages: very little memory, fast, and easy to run continuously as traffic changes.

Limitations:

- It gets stuck in local optima, plateaus and ridges (next answer).
- The result depends on the starting plan.
- It never accepts a worse move, so it cannot escape a poor solution.
- It is not complete or guaranteed optimal.

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

Answer:

- **Local maximum (local optimum):** a timing plan where every small change makes things worse, yet a far better plan exists. Example: one main road is perfectly tuned for a green wave, but any single change would hurt it, even though a different plan for the whole corridor would cut waiting time further.
- **Plateau:** a region where many neighbouring plans give almost the same total waiting time (e.g. during steady off-peak flow), so the algorithm has no direction and wanders or stops.
- **Ridge:** a narrow path of improvement that needs several parameters to change together, e.g. shifting the offset at intersection A and extending green at intersection B simultaneously. Changing only one parameter at a time lowers the score, so single-step moves cannot climb the ridge.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

Answer:

- **Simulated Annealing (SA):** accepts a worse plan with probability e^(−ΔE/T), where ΔE is the increase in cost and T is the "temperature". Early on (high T) it explores widely and can escape local optima; as T cools it becomes more selective and converges on a good plan. It handles plateaus by random moves and ridges by occasionally taking a temporary downhill step.
- **Genetic Algorithms (GA):** keep a population of timing plans. Fitness is the inverse of cost. Selection keeps good plans, crossover combines good parts of two plans (e.g. corridor timings from different parents) and mutation adds random changes. Searching from many points in parallel avoids being trapped and can find better combinations of corridor settings, including ridge-like ones.
- Both accept temporary or varied solutions, so they explore the space broadly instead of only climbing the nearest hill.

iv. Recommend the best approach and justify your choice based on scalability and adaptability

Answer:

Recommendation: a hybrid approach, Genetic Algorithm (with traffic simulation for fitness) for global planning, plus Simulated Annealing or hill climbing for fast local fine-tuning.

- **Scalability:** GA evaluates many candidate plans in parallel, which fits hundreds of intersections and can be split by district or corridor.
- **Adaptability:** the population can be kept and re-evolved as traffic patterns change (rush hour, accidents, events), so there is no need to restart from scratch. SA or hill climbing then adjusts live in real time.
- **Quality:** it avoids local optima far better than plain hill climbing and gives high-quality, though not provably optimal, solutions, which is acceptable for city traffic.
- If computing resources are limited, SA alone is the lightest option with good results.

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

Answer:

- An **offline** agent computes a complete plan before acting, which requires a full map and a predictable environment. The rover has neither: the terrain is unknown and craters and rocks are discovered only when sensed.
- An **online search agent** interleaves planning and acting: it takes a step, senses, updates its knowledge and then decides the next step.
- Online search is needed because (1) the state space is not known in advance, (2) actions have real consequences, so mistakes can be fatal (falling into a crater), (3) Earth–Mars communication delay (minutes) makes remote control impractical, so the rover must decide on its own, and (4) the environment may change (sand shifts, new hazards appear).

ii. Analyze the challenges of partial observability and dynamic environments

Answer:

- **Partial observability:** sensors (cameras, lidar) see only a limited range and may be affected by dust, shadows, or noise, so the rover never knows the full state. It may mistake a shadow for a crater or miss a small rock.
- **Dynamic environment:** terrain can change (loose soil, dust storms, slipping slopes), so earlier knowledge may become outdated.

- **Other challenges:** localisation error (wheel slip, drift), limited battery and computing power, irreversible actions (getting stuck), and the need to avoid revisiting dead ends and loops.
- **Handling these:** keep a belief about the state, treat unknown cells cautiously, use conservative safety margins, and re-plan whenever new observations arrive.

iii. Describe how the rover builds and updates its internal model

Answer:

- The rover starts with an empty or minimal map, its position, and its goal (sample site).
- At each step it senses its surroundings and records them in an internal model such as an **occupancy/cost grid**, marking cells as free, obstacle (crater, rock), or unknown, with a traversability cost for slopes and soft soil.
- It stores **results of its actions**, e.g. result[state, action] = new state, and records visited states and dead ends so it does not repeat them.
- It uses odometry and landmark matching (SLAM) to keep its position estimate and map consistent.
- When new data conflicts with the old (a path now blocked), the model is updated and the plan is revised. Learned heuristic values (as in LRTA*) improve over repeated visits.

iv. Compare online search with uninformed and informed search strategies

Answer:

- **Uninformed (BFS, DFS, uniform-cost):** need a known state space and use no goal knowledge. Offline versions are impractical here, and online versions such as online DFS would explore blindly, wasting time and battery, though they are simple and complete for finite spaces with backtracking.
- **Informed (Greedy, A*):** use a heuristic such as distance to the sample site, so they are more efficient. However, standard A* assumes a known map; applied offline it breaks when unexpected obstacles appear.
- **Online search (e.g. online DFS, LRTA*, D* Lite):** works with what is currently known, explores locally, learns from experience and can be combined with a heuristic. Its drawbacks are that it cannot guarantee an optimal path and may need to backtrack. It is the only type that fits unknown terrain.
- In short: uninformed = blind but simple; informed = efficient but needs a map; online = adaptive and practical for unknown environments.

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

Answer:

Recommendation: an online informed search, specifically LRTA* / D* Lite style planning with a hazard-aware heuristic, supported by a continuously updated map.

- **Uncertainty:** it plans only with known information and updates the plan with each new observation.
- **Adaptability:** D* Lite re-plans efficiently by repairing only the affected part of the path, and LRTA* improves its heuristic estimates as it learns.
- **Safety:** unknown cells receive high costs, obstacles are given safety margins, and the rover prefers reversible moves. If no safe path exists it waits or returns to a known safe location.
- **Efficiency:** the heuristic (distance to the sample site) keeps exploration directed and saves battery, much better than blind exploration.

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

Answer:

A CSP is defined by a set of variables, a domain for each variable, and a set of constraints. Here:

- **Goal:** assign every exam a time slot and an exam hall so that all constraints are satisfied.
- **Formulation:** X = {exam for each course}, D = {possible (time slot, hall) pairs}, C = {constraints below}.
- A complete, consistent assignment is a valid timetable. If we want the best one, an optimisation objective (e.g. minimise the number of days or gaps between exams) can be added on top (a constraint optimisation problem).

i. Clearly define variables, domains, and constraints with explanation

Answer:

Variables: one variable per course exam, E_1, E_2, … E_n (optionally also Hall(E_i) and Invigilator(E_i)).

Domains: Dom(E_i) = {(day, time slot, hall)}, e.g. 10 days × 3 slots × available halls. Domains are reduced in advance for special cases (e.g. a faculty member unavailable on certain days).

Constraints:

- **Student clash (binary):** if courses E_i and E_j share at least one student, then slot(E_i) ≠ slot(E_j). This stops overlapping exams.
- **Hall capacity (global/resource):** the exams in the same slot must not exceed the number of halls, and the students in a hall must not exceed its capacity.

- **Precedence (binary):** if subject A must be scheduled before subject B, then slot(A) < slot(B).
- **Faculty availability (unary):** the invigilator/examiner must be available in the chosen slot, so slots when they are unavailable are removed from the domain.
- **Special accommodations (unary/binary):** students needing extra time or a separate room force a specific hall type or a longer slot; assigned halls must be accessible.

Hard constraints must always hold; soft constraints (e.g. not more than two exams per day for a student) are preferences that are optimised.

ii. Explain how backtracking search works in this scenario

Answer:

- Pick an unassigned exam variable, try a value (slot and hall) from its domain, and check it against the constraints with the exams already scheduled.
- If it is consistent, assign it and move on to the next variable. If no value works, **backtrack**: undo the previous assignment and try another value.
- Continue until all exams are assigned (success) or all options are exhausted (failure).
- Example: schedule Maths on Day 1 morning in Hall A; then Physics (shares students with Maths) cannot take the same slot, so try Day 1 afternoon. If a later exam has no valid slot, return and change an earlier choice.
- Plain backtracking is complete, but it can be very slow because it discovers conflicts late. Heuristics improve it: **MRV** (choose the variable with the fewest remaining values), **degree heuristic** (most constrained exam first) and **LCV** (try the value that rules out the fewest options for others).

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

Answer:

- **Forward checking:** after assigning an exam to a slot, remove that slot from the domains of all unassigned exams that conflict with it (e.g. exams sharing students). If any domain becomes empty, the search backtracks immediately instead of continuing down a doomed branch.
- This detects failures early, avoids exploring huge parts of the search tree, and works well with MRV, since domains are kept up to date.
- **Stronger propagation (arc consistency, AC-3):** also checks that every remaining value of a variable has a compatible value in each neighbour's domain, pruning more values (e.g. enforcing precedence A before B shrinks the domains of both).
- The result is far fewer backtracks and much faster timetables, at a small extra cost per step.

iv. Analyze real-world challenges and justify the best strategy for scalability

Answer:

Challenges:

- Thousands of courses and students create a huge number of constraints, and the problem is NP-hard, so exhaustive search does not scale.
- Conflicting requirements may make a perfect solution impossible, so soft constraints and priorities are needed.
- Last-minute changes (room unavailable, new accommodation request) require re-scheduling with minimal disruption.
- Fairness and student wellbeing (gaps, exams per day) are hard to express precisely.

Best strategy: a combined approach:

- Model as a CSP and use backtracking with **MRV + degree + LCV heuristics, forward checking / AC-3** to handle hard constraints efficiently.
- Use **local search (min-conflicts) or simulated annealing** to repair and improve the timetable for soft constraints and for quick updates when something changes.
- Decompose the problem (by faculty or day) and keep a human in the loop for exceptions.

This gives valid timetables quickly, scales to large universities and adapts to change.

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

Answer:

- Minimax models the game as a two-player, zero-sum adversarial search tree. Nodes are game states, edges are legal moves, and leaves (or the depth limit) get a utility value.
- **MAX** is the AI, which picks the move with the highest value. **MIN** is the human opponent, assumed to play the best counter-move that minimises the AI's value.
- Values are backed up from the leaves: a MAX node takes the maximum of its children, a MIN node the minimum. At the root, the AI plays the move with the best guaranteed value against optimal play.
- In a real-time strategy game the AI evaluates its possible actions (build, attack, defend) against the human's possible replies, assuming the human responds optimally, giving a safe, worst-case-aware decision.

ii. Analyze the computational challenges of large game trees

Answer:

- The tree size is about $\mathbf{b^d}$, where b is the branching factor (number of possible actions) and d is the depth. In RTS games b can be in the hundreds or more, since many units can each act, so even a few moves ahead means billions of states.
- Time: decisions must be made within milliseconds to seconds, while full minimax is far too slow.
- Memory: storing and evaluating all states is expensive.
- Extra difficulties: real-time play (the game does not wait), imperfect information (fog of war), chance elements, and the need for a good evaluation function when the search cannot reach terminal states.
- Hence depth limits, pruning, move ordering, and heuristics are essential.

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

Answer:

- Alpha-Beta pruning gives **exactly the same result as minimax** but skips branches that cannot affect the final decision.
- It keeps two bounds: $\boldsymbol{\alpha}$ = the best value MAX can already guarantee along the path, and $\boldsymbol{\beta}$ = the best value MIN can already guarantee. If at any node $\boldsymbol{\alpha \geq \beta}$, the remaining children can be pruned.
- Example: MAX has found a move worth 5 ($\alpha = 5$). While examining another move, MIN finds a reply worth 3. MIN would pick 3 or less, which is worse than 5 for MAX, so the other replies of that move need not be examined.
- **Efficiency:** with random ordering it examines about O(b^(3d/4)) nodes; with best-case move ordering it reduces the work from $O(b^d)$ to about **O(b^(d/2))**, which means it can search roughly **twice as deep** in the same time.
- Good move ordering (try captures, strong attacks and previous best moves first) is the key to maximum pruning.

iv. Design an evaluation function and justify the factors considered

Answer:

When the search stops at a depth limit, a heuristic **evaluation function** estimates how good a state is for the AI. A weighted linear form works well:

$\mathbf{Eval(s) = w_1 \cdot Resources + w_2 \cdot Army\ strength + w_3 \cdot Territory/Map\ control + w_4 \cdot Tech/Upgrades + w_5 \cdot Economy + w_6 \cdot Safety - w_7 \cdot Enemy\ strength}$

- **Army strength (unit count × unit quality/health):** the direct measure of combat power.
- **Resources and economy (income rate, workers):** long-term ability to produce units.
- **Map/territory control and vision:** key positions give tactical advantages and information.
- **Technology/upgrades:** future potential of the army.

- **Base safety (health of buildings, defences, threats nearby):** protects against losing the game.
- **Enemy's values (negative terms):** the advantage is relative, so we subtract the opponent's strength.
- Winning states get $+\infty$ and losing states $-\infty$. Weights are tuned from expert knowledge, self-play or machine learning. The function must be **fast to compute** and correlate well with the actual chance of winning.

v. Recommend a strategy for balancing optimality and real-time performance

Answer:

Recommendation: Alpha-Beta search with iterative deepening and a time limit, supported by heuristics.

- **Iterative deepening:** search depth 1, then 2, 3, … until the time budget ends, and then play the best move from the last completed depth. This guarantees an answer on time and improves move ordering for the next round.
- **Move ordering and transposition tables:** try the most promising moves first for stronger pruning, and cache repeated states.
- **Depth limit + evaluation function:** stop at a fixed depth and use the evaluation function above. Use quiescence search so that unstable (battle) positions are searched deeper.
- **Selective pruning and abstraction:** prune clearly bad moves (beam search), group units into squads so that the branching factor stays low.
- **Alternative for huge trees:** Monte Carlo Tree Search can be used for very large branching factors.

This gives strong play that is near-optimal for the available time while always responding in real time.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand